

PROVIDER_FEATURES

Provider Feature Matrix

This document provides a comprehensive overview of which advanced features are supported by each LLM provider in HelixCode.

Feature Support Matrix

Provider	Prompt Caching	Reasoning/Thinking	Token Budgets	Notes
Anthropic	✔ Full	✔ Extended Thinking	✔ Full	Supports ephemeral cache, thinking budgets
OpenAI	✔ Automatic	✔ O-series (o1/o3)	✔ Full	Prompt caching auto-enabled since Oct 2024; reasoning on o-series
Azure	✔ Automatic	✔ O-series support	✔ Full	Prompt caching enabled by default on GPT-4o+ (Azure OpenAI / Foundry)
Gemini	✔ Context caching	✔ Native reasoning	✔ Full	Implicit caching default on Gemini 2.5+; explicit caching available
VertexAI	⚠ Via Gemini	✔ Via Gemini	✔ Full	GCP-hosted AI models
Bedrock	⚠ Via provider	✔ Model-dependent	✔ Full	AWS-hosted models

Provider	Prompt Caching	Reasoning/Thinking	Token Budgets	Notes
				(Claude, etc.)
Groq	⚠ Automatic (subset)	✅ Model-dependent	✅ Full	Prompt caching auto on GPT-OSS models (50% cached discount)
Qwen	❌ Not supported	✅ QwQ-32B	✅ Full	Alibaba Cloud models
xAI	❌ Not supported	✅ Model-dependent	✅ Full	Grok models
OpenRouter	⚠ Via upstream	✅ Via upstream	✅ Full	Aggregator, feature depends on model
Copilot	❌ Not supported	✅ GPT-4 based	✅ Full	GitHub Copilot integration
Local	❌ Not applicable	✅ Model-dependent	✅ Full	Ollama, llama.cpp

Legend: -  **Full:** Feature is fully supported with native implementation -  **Planned/Limited:** Feature is planned or has limited support -  **Not supported:** Feature is not available for this provider - **Via:** Feature available through underlying provider

Feature Details

1. Prompt Caching

Prompt caching reduces costs by caching frequently used context (system prompts, tool definitions, conversation history).

Anthropic Claude

- **Support:** Full native support via `cache_control` directive
- **Strategies:** System, Tools, Context, Aggressive
- **Cost Savings:** Up to 90% reduction on cached input tokens
- **Configuration:**

```
cacheConfig := &llm.CacheConfig{
    Enabled: true,
```

```

Strategy: llm.CacheStrategyTools,
MinTokensForCache: 1024,
CacheTTL: 300, // 5 minutes
}

```

- **Minimum tokens:** model-dependent per Anthropic’s official docs — 1,024 tokens for Claude Sonnet 4.x / Opus 4.1 / earlier; 4,096 tokens for the newest Opus and Haiku 4.5 models. The `MinTokensForCache: 1024` example above is the lower bound and may be rejected by the newest models.
- **TTL:** 5 minutes default; an optional 1-hour TTL is available at 2x base input-token price (`{"cache_control":{"type":"ephemeral","ttl":"1h"}}`)

OpenAI

- **Support:** Automatic prompt caching (enabled by default, no opt-out) for prompts $\geq 1,024$ tokens with an identical leading prefix; cache hits reported as `cached_tokens` under `prompt_tokens_details`. Released October 2024.
- **Note:** Caching is automatic — no `cache_control` directive is required (unlike Anthropic). The `HelixCode CacheConfig` integration is therefore advisory for OpenAI (prefix structuring) rather than an explicit cache directive.

Gemini/VertexAI

- **Support:** Two mechanisms per Google’s official docs — *implicit caching* (enabled by default on Gemini 2.5 and newer, no developer action) and *explicit caching* (developer-managed cached content with a configurable TTL, default 1 hour)
- **Configuration:** Explicit caching via API; implicit caching automatic
- **Cost:** Reduced pricing for cached context

Azure OpenAI

- **Support:** Prompt caching enabled by default (no opt-out) for all GPT-4o-or-newer models; minimum 1,024 tokens with identical first-1,024-token prefix. In-memory retention cleared within 5–10 min of inactivity ($\leq 1h$); extended retention (up to 24h) available on GPT-5.x / GPT-4.1 family.
- **Status:** Generally available (Microsoft Foundry / Azure OpenAI)

2. Reasoning/Thinking Mode

Advanced reasoning capabilities for complex problem-solving tasks.

Anthropic Claude

- **Support:** Extended thinking mode (claude-4-sonnet, claude-3-7-sonnet)
- **Configuration:**

```

reasoningConfig := &llm.ReasoningConfig{
    Enabled: true,
    ThinkingBudget: 5000, // tokens for thinking
    ExtractThinking: true,
}

```

```

    HideFromUser: false,
    ReasoningEffort: "medium", // low, medium, high
}

```

- **Thinking Budget:** Separate token allocation for internal reasoning
- **Auto-detection:** Enabled automatically for thinking-related keywords
- **Extraction:** Thinking blocks can be separated from final output

OpenAI O-Series

- **Models:** o1, o1-preview, o1-mini, o3, o4 (planned)
- **Support:** Native reasoning models with automatic thinking
- **Configuration:**

```

reasoningConfig :=
    llm.NewReasoningConfig(llm.ReasoningModelOpenAI_01)

```

- **Thinking tokens:** Counted separately in usage
- **Cost:** Higher per-token cost for reasoning

DeepSeek R1

- **Support:** Via OpenRouter or compatible APIs
- **Thinking tags:** Uses <think> tags for reasoning
- **Configuration:** Automatic when model name contains “deepseek” and “r1”

QwQ-32B (Qwen)

- **Support:** Dedicated reasoning model
- **Configuration:** Auto-detected by model name
- **Budget:** 7000 tokens default for thinking

3. Token Budgets

Comprehensive token and cost tracking across all providers.

Features

- **Per-request limits:** Maximum tokens per single request
- **Session limits:** Total tokens across a session
- **Cost tracking:** USD cost estimation per provider
- **Rate limiting:** Requests per minute throttling
- **Warning thresholds:** Configurable alerts at X% usage

Configuration

```
budget := llm.TokenBudget{
    MaxTokensPerRequest: 10000,
    MaxTokensPerSession: 100000,
    MaxCostPerSession: 10.0, // $10 USD
    MaxCostPerDay: 50.0,    // $50 USD
    MaxRequestsPerMinute: 60,
    WarnThreshold: 80.0,    // Warn at 80%
}

pm := llm.NewProviderManagerWithBudget(config, budget)
```

Usage Tracking

```
// Get session usage
usage, err := pm.GetSessionUsage(sessionID)
fmt.Printf("Tokens used: %d / %d\n", usage.TotalTokens,
    budget.MaxTokensPerSession)
fmt.Printf("Cost: $%.2f / $%.2f\n", usage.TotalCost,
    budget.MaxCostPerSession)

// Get budget status
status := pm.GetBudgetStatus(sessionID)
fmt.Printf("Token usage: %.1f%%\n", status.TokenUsagePercent)
fmt.Printf("Cost usage: %.1f%%\n", status.CostUsagePercent)
```

Cost Estimation

Costs are estimated based on published provider pricing: - **Anthropic Claude 3.5 Sonnet**: \$3/\$15 per 1M tokens (input/output) - **OpenAI GPT-4o**: \$2.50/\$10 per 1M tokens - **OpenAI O1**: \$15/\$60 per 1M tokens (with thinking) - **Gemini Pro**: \$0.50/\$1.50 per 1M tokens - **Groq**: Often free tier available, then \$0.10/\$0.10 per 1M tokens

Implementation Guide

Updating LLMRequest

All new features are integrated via the `LLMRequest` structure:

```
request := &llm.LLMRequest{
    Model: "claude-3-5-sonnet-20241022",
    Messages: messages,
    MaxTokens: 4096,

    // Reasoning configuration
    Reasoning: &llm.ReasoningConfig{
        Enabled: true,
        ThinkingBudget: 5000,
    },
}
```

```

        ReasoningEffort: "medium",
    },

    // Cache configuration
    CacheConfig: &llm.CacheConfig{
        Enabled: true,
        Strategy: llm.CacheStrategyTools,
    },

    // Token budget (set session ID for tracking)
    SessionID: sessionID,
    ThinkingBudget: 5000, // Override reasoning budget
}

response, err := providerManager.Generate(ctx, request)

```

Auto-Detection

HelixCode automatically detects reasoning models and applies appropriate configurations:

```

// These model names trigger automatic reasoning mode:
// - OpenAI: "o1", "o1-preview", "o1-mini", "o3", "o4"
// - Anthropic: "claude-4-sonnet", "claude-3-7-sonnet", "opus"
//           (with keywords)
// - DeepSeek: "deepseek-r1", "deepseek-reasoner"
// - Qwen: "qwq-32b"

request := &llm.LLMRequest{
    Model: "o1-preview", // Automatically enables reasoning
    Messages: messages,
}

```

Cache Strategy Selection

Choose caching strategy based on use case:

- **CacheStrategyNone:** Disable caching
- **CacheStrategySystem:** Cache only system messages
- **CacheStrategyTools:** Cache system + tool definitions (recommended)
- **CacheStrategyContext:** Cache system + tools + recent messages
- **CacheStrategyAggressive:** Cache everything possible

Budget Management

```

// Initialize with budget
pm := llm.NewProviderManagerWithBudget(config,
    llm.DefaultTokenBudget())

// Track requests
request.SessionID = "user-session-123"

```

```

response, err := pm.Generate(ctx, request)

// Monitor usage
if status := pm.GetBudgetStatus(request.SessionID); status !=
    nil {
    if status.TokenUsagePercent > 90 {
        log.Printf("Warning: %s approaching token limit",
            request.SessionID)
    }
}

// Reset session when done
pm.ResetSession(request.SessionID)

```

Testing

Integration tests verify feature support across providers:

```

cd HelixCode
go test -v ./internal/llm -run TestReasoningFeatures
go test -v ./internal/llm -run TestCacheControl
go test -v ./internal/llm -run TestTokenBudgets

```

Migration Notes

Existing Code

Existing code continues to work without changes:

```

// Old code - still works
request := &llm.LLMRequest{
    Model: "claude-3-5-sonnet-20241022",
    Messages: messages,
    MaxTokens: 4096,
}
response, err := provider.Generate(ctx, request)

```

Opt-in Features

New features are opt-in via configuration:

```

// New code - with advanced features
request := &llm.LLMRequest{
    Model: "claude-3-5-sonnet-20241022",
    Messages: messages,
    MaxTokens: 4096,
    Reasoning:
        llm.NewReasoningConfig(llm.ReasoningModelClaude_Sonnet),
}

```

```
CacheConfig: &llm.CacheConfig{Enabled: true, Strategy:
    llm.CacheStrategyTools},
SessionID: sessionID,
}
```

Performance Impact

Prompt Caching

- **First request:** ~5-10ms overhead to create cache
- **Cached requests:** 2-5x faster response time
- **Cost savings:** 80-90% on cached input tokens
- **Best for:** Multi-turn conversations, repeated system prompts

Reasoning Mode

- **Latency:** 2-3x longer due to thinking phase
- **Quality:** Significantly better for complex tasks
- **Cost:** Higher token usage (thinking + output)
- **Best for:** Planning, analysis, complex problem-solving

Token Budgets

- **Overhead:** <1ms per request for tracking
- **Memory:** ~1KB per session
- **Best for:** Multi-user systems, cost control

Roadmap

Planned Improvements

1. **Anthropic**
 - ☒ Full prompt caching
 - ☒ Extended thinking
 - ☐ Batch API support
 - ☐ Tool caching optimization
2. **OpenAI**
 - ☐ Prompt caching (when available)
 - ☒ O-series reasoning
 - ☐ Structured outputs
 - ☐ Batch API integration
3. **Cross-Provider**
 - ☐ Unified caching layer
 - ☐ Cost optimization engine
 - ☐ Automatic failover with feature preservation
 - ☐ Provider-agnostic reasoning interface

Experimental Features

- **Reasoning trace visualization:** Display thinking process to users
- **Dynamic budget allocation:** Adjust budgets based on task complexity

- **Multi-provider reasoning:** Combine reasoning from multiple models
- **Cache analytics dashboard:** Visualize cache hit rates and savings

Support

For issues or questions about provider features: - Create an issue on GitHub - Check the integration tests for examples - Review provider-specific documentation in `/docs/providers/`

References

- [Anthropic Prompt Caching](#)
- [OpenAI O-Series Models](#)
- [Google Gemini Context Caching](#)
- [AWS Bedrock Documentation](#)

Sources verified

Sources verified 2026-05-29: <https://platform.claude.com/docs/en/docs/build-with-claude/prompt-caching> ; <https://ai.google.dev/gemini-api/docs/caching> ; <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/how-to/prompt-caching> ; <https://console.groq.com/docs/prompt-caching> ; https://api-docs.deepseek.com/guides/reasoning_model ; <https://github.com/ollama/ollama/blob/main/docs/api.md> ; <https://docs.x.ai/docs/models> — Confirmed against official sources on 2026-05-29: (1) Anthropic `cache_control` prompt caching with model-dependent minimum (1,024 tokens for Sonnet 4.x/Opus 4.1/earlier; 4,096 for newest Opus and Haiku 4.5), default 5-min TTL + optional 1-h TTL at 2x price — the prior blanket “1024 minimum” was corrected. (2) Gemini implicit caching default on 2.5+ and explicit caching (default 1-h TTL). (3) Azure OpenAI prompt caching is enabled-by-default on GPT-4o+ (was wrongly listed “Planned”). (4) DeepSeek `deepseek-reasoner` exposes `reasoning_content` CoT — matches doc. (5) Ollama `/api/tags + think` parameter confirmed. (6) xAI Grok reasoning + 1M-token context confirmed.

Negative findings / could-not-verify (§11.4.99(B)): - **OpenAI official prompt-caching docs unreachable** — both <https://platform.openai.com/docs/guides/prompt-caching> and <https://openai.com/index/api-prompt-caching/> returned HTTP 403 to the fetcher. The OpenAI/Azure automatic-caching claims here are corroborated from the official **Azure OpenAI** (Microsoft Learn) page (`prompt_cache_retention`, `cached_tokens`, “enabled by default”, 1,024-token minimum) and Anthropic’s own multi-cloud doc, NOT from a directly-fetched openai.com page. Treat the OpenAI-platform-specific wording as cross-corroborated, not first-party-confirmed in this session. - **Groq prompt caching is model-scoped** — Groq’s official docs confirm automatic caching ONLY on GPT-OSS 20B/120B + GPT-OSS-Safeguard 20B (50% cached-token discount, ~2-h volatile retention), NOT on all Groq models. The prior matrix entry “ Not supported” was stale; corrected to “ Automatic (subset)”. - **xAI prompt caching: not documented** — docs.x.ai/docs/models does not address prompt caching, so the matrix “ Not supported” for xAI is unconfirmed-either-way (absence of documentation, not a confirmed negative). - **AWS Bedrock page body not extractable** — <https://docs.aws.amazon.com/bedrock/latest/userguide/inference-prompt-caching.html> returned only the page title to the fetcher (JS-rendered body not captured); Bedrock-as-a-caching-surface is corroborated only indirectly via Anthropic’s multi-cloud doc.

CONST-036/037 caveat: This matrix hardcodes per-provider capability claims. Per CONST-036/037, capability flags MUST be sourced at runtime from LLMsVerifier (VerificationResult), not from a static doc. This file is descriptive/operator-facing reference only and MUST NOT be used as the runtime source of truth for capability gating; treat any divergence between this matrix and LLMsVerifier as LLMsVerifier-wins.